“Major overhaul” change overview

1. Project Structure & Tooling

To uphold ***SOLID*** principles and the ***Divide-and-Conquer*** approach, the project has been split into distinct namespaces and directories:

```
mimiru_db/
├── CMakeLists.txt          # Modern, target-based CMake configuration
├── .clang-format           # Global C++ styling rules
├── include/mimiru/         # Public API headers (.hpp)
│   ├── core/               # Constants, Types, Enums, Exceptions
│   ├── storage/            # Disk I/O (Pager, String Interner)
│   ├── index/              # B+ Tree
│   └── engine/             # Table, Database, SystemManager
├── src/                    # Implementation files (.cpp)
│   ├── storage/
│   ├── index/
│   └── engine/
└── tests/                  # Google Test suites
    ├── CMakeLists.txt
    ├── test_storage.cpp
    ├── test_bplus_tree.cpp
    └── test_engine.cpp
```

1.1 Tooling: CMake & GTest

We transitioned from simple ***assert()*** statements to ***Google Test (GTest)***. GTest provides test fixtures (SetUp/TearDown), allowing us to cleanly generate and delete test database files before and after every single test. CMake is now configured to fetch GTest and ***nlohmann_json*** (required for JSON output) automatically.

1.2 Tooling: .clang-format

We adopted the Google C++ Style Guide via ***.clang-format***.

2. Core Architectural Shifts

2.1 Moving Away from Crude Memory Offsets

Previously, we defined binary layouts using hardcoded integer offsets (e.g., ***buffer[NODE_TYPE_OFFSET]***). This is extremely error-prone and hard to maintain.

The New Approach: ***Packed Struct Overlays***

We now use C++ structs mapped directly onto raw 4KB memory buffers using ***reinterpret_cast***.

To prevent the C++ compiler from destroying our binary layout by adding invisible padding bytes, we wrap all disk-bound structs in compiler directives:

```
#pragma pack(push, 1) // Force 1-byte alignment (Zero padding)
struct NodeHeader {
   NodeType type;            // 1 byte
   uint8_t is_root;          // 1 byte
   core::PageId parent_page;   // 4 bytes
   uint32_t count;           // 4 bytes
}; // Total: Exactly 10 bytes on all platforms
#pragma pack(pop)
```

Why this is better: We can access data natively as ***header->count*** instead of doing bit-shifting math, while retaining perfect binary precision on the disk.

2.2 Eliminating In-Memory Pointers

A major shift was refactoring the B+ Tree from an ***In-Memory structure*** to an ***On-Disk structure***.

Old: Nodes held C++ ***pointers (Node* left_child)***. If the DB restarts, 0x7ffee... is garbage memory.

New: Nodes hold ***PageId (uint32_t)***. A PageId is a relative offset (e.g., Page 5 is exactly at byte 5 * 4096). This data safely survives reboots (fulfilling the Reliability constraint).

2.3 String Interning & Disk Safety

Because std::string uses heap pointers, it cannot be written directly to disk via ***struct overlays***. To comply with the ***String Interning*** requirement:

Strings are saved in a separate dictionary file.
The dictionary hands back a ***uint32_t StringId***.
Our tables and indices only ever store this 4-byte ***StringId***.

3. Module Breakdown & Functionality

mimiru::core (Types & Metadata)

Contains ***types.hpp***. Defines physical constraints (***PAGE_SIZE***), primitives (***PageId***), and ***Schema definitions***. It also handles the conversion of internal Record objects into JSON arrays using C++17 std::variant and ***nlohmann/json***.

mimiru::storage (Disk I/O)

Pager: The lowest-level system component. It knows nothing about databases or trees. Its sole job is moving fixed 4096-byte blocks (***pages***) between ***RAM*** and the ***Hard Drive***. It manages file streams and handles appending new pages.

mimiru::index (B+ Tree)

BPlusTreeInt: A fully persistent ***B+ Tree*** mapped onto 4KB pages.
Recursive Splitting: Handles in-memory overflow before perfectly splitting nodes in half and pushing keys up to the parent.
Doubly-Linked Leaves: Leaf nodes maintain ***next_page*** and ***prev_page*** pointers. This allows O(1) horizontal traversal for SQL ***BETWEEN*** (***Range***) queries, bypassing the tree structure completely during iteration.

mimiru::engine (Table & Execution)

Table: Manages the logical rows of a specific schema.
Superblock (Page 0): The first page of every ***.db*** file is reserved. It holds pointers to the start of the data and the root of the index, preventing file corruption.
Pagination: Implements a ***Slotted Page design***. When a 4KB page fills up with records, the ***Table*** dynamically allocates a new page and links it to the previous one.
Constraint Validation: Checks ***NOT_NULL*** and ***INDEXED*** constraints before allowing insertions.
Database & SystemManager: Manages the operating system directory tree.

SystemManager controls the *root mimiru_data/ folder* and *CREATE DATABASE / USE logic*.
Database controls individual folders and handles *CREATE TABLE logic*, creating separate *.db* files and wiring them to *Pager* instances.

4. The Database Pipeline (Data Flow)

Here is exactly what happens when the engine processes an insertion (e.g., *INSERT INTO users (id, price) VALUES (101, 500)*):
System routing: The *SystemManager* forwards the request to the currently active *Database*.
Table routing: The *Database* looks up the *Table* object for *"users"*.
Constraint Check: The *Table* validates the row. If id is marked *INDEXED*, it queries the *BPlusTreeInt* to ensure 101 doesn't already exist.
Pagination: The *Table* asks the *Pager* for the last known data page. If the page is full, it tells the *Pager* to allocate a new one.
Disk Overlay: The *Table* maps the page buffer to a *PageHeader struct*, advances the free space offset, and copies the *(101, 500)* integers directly into the binary slot.
*Index Updat*e: The Table bit-packs the PageId and SlotId into a single *32-bit RecordId* and passes it to the *BPlusTreeInt*.
Disk Sync: The *Table* asks the *Pager* to flush the updated *Data Page* and the updated *Index Pages* back to the hard drive.